



ENGINEERING

CIS*6560*01: Cybersecurity Project -
Summer 2024

Topic: Secure Federated Learning with Differential
Privacy

Professor: Rozita Dara

By
Ripunjy Madhab Buddha
Student ID: 1272183

Table of Contents

S. No	Contents	Page
1.	Introduction	1 - 3
2.	Objective	3
3	Motivation and Related works	3 - 5
	I. Defense against cyber attacks	3
	II. Related works	5
4	Implementation	5 - 12
	I. Data preparation	6
	II. Initiate server	7
	III. Define model	8
	IV. Initiate client	10
	V. Incorporate differential privacy	11
	VI. Train model	12
5	Result	13
6	Challenges	14 - 15
7	Conclusion	15

1. Introduction

In traditional centralized machine learning, the model training process requires aggregating vast amounts of data from various sources into a single location. This centralized approach presents several critical challenges. The most serious of these concerns is data privacy and security. When sensitive data, such as personal information or proprietary business data, is centralized, it becomes a prime target for cyber-attacks and data breaches. The risk of unauthorized access to this data can lead to significant financial loss, legal repercussions, and damage to reputation. Moreover, strict data protection regulations like the General Data Protection Regulation (GDPR)¹ and the Health Insurance Portability and Accountability Act (HIPAA)² impose requirements on how data can be collected, stored, and shared, making compliance a complex and costly process.

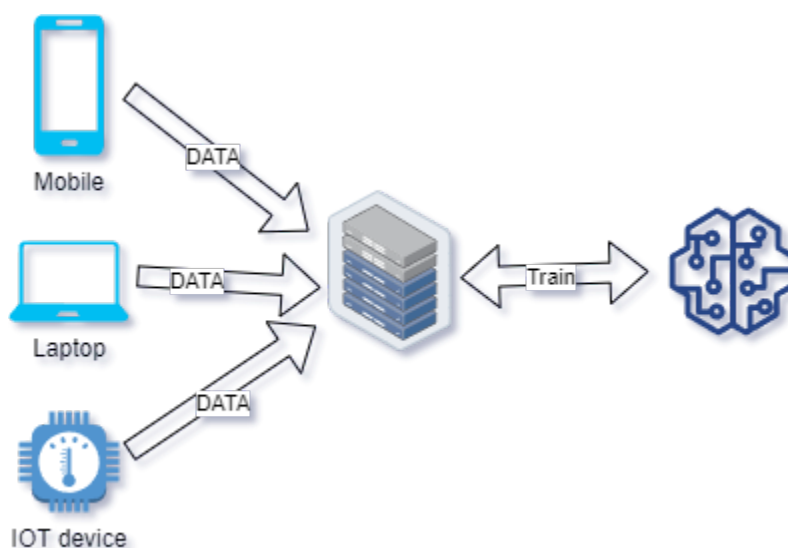


Fig- 1.1 Centralized machine learning

Federated learning solves these problems by offering a solution to these challenges by enabling the training of machine learning models in a decentralized manner. In this approach, data remains on local devices, and only model updates—such as gradients

¹ Art. 50 GDPR – International cooperation for the protection of personal data - General Data Protection Regulation (GDPR). (2016, October 6). General Data Protection Regulation (GDPR). <https://gdpr-info.eu/art-50-gdpr/>

² Rights, O. F. C. (2022, October 19). *Summary of the HIPAA Privacy Rule*. HHS.gov. <https://www.hhs.gov/hipaa/for-professionals/privacy/laws-regulations/index.html>

or weights—are transmitted to a central server. This decentralized framework significantly enhances data privacy and security, as the raw data never leaves the local devices, thereby reducing the risk of breaches and aiding compliance with data protection regulations.

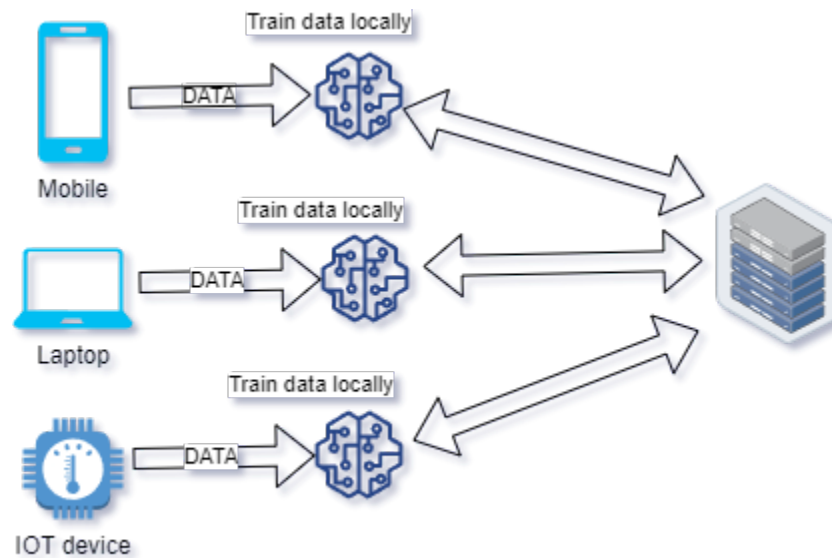


Fig-1.2 Federated machine learning

Centralizing the data is not only the problem. Another key problem is that model training may memorize and leak sensitive data, as trained models retain and share exact features from the training data, providing further privacy risks.

To solve this problem, federated learning is incorporated with **differential privacy** techniques. Differential privacy provides a mathematical framework to ensure that the inclusion or exclusion of a single data point does not significantly affect the outcome of the analysis, thereby preserving the privacy of individual data entries. In the context of federated learning, differential privacy can be applied to the model updates being shared, adding noise to these updates to prevent the extraction of sensitive information. This technique further enhances the privacy guarantees of federated learning by ensuring that even if the model updates are intercepted or analyzed, the privacy of the underlying data remains intact.

2. Objective

The objective of this project is to develop and demonstrate a secure federated learning framework that leverages differential privacy (DP) to enhance data privacy and security during the collaborative training of machine learning models. The system will allow multiple decentralized parties or clients to jointly train a global model without sharing their raw data, thereby preserving the privacy and confidentiality of sensitive information.

3. Motivation and Related works

In today's digital age, securing data is becoming increasingly challenging, especially as large corporations rely on user data to train their models, creating significant risks to data privacy. The growing number of cyberattacks worsens this issue, putting sensitive data at even greater risk. My main motivation for this project is to explore methods that can help defend against these cyberattacks.

1. Defense against cyber attacks

Federated learning and differential privacy together offer a powerful defense against various cyber-attacks, ensuring data privacy and security in machine learning applications. Federated learning addresses the inherent risks of centralized data storage. Complementing this, differential privacy introduces noise into the data and model updates. This obfuscation is crucial in defending against various critical cyber attacks.

Some cyber attacks and how federated learning and differential privacy help protect against them are discussed below:

I. Data Breaches: Data breaches involve unauthorized access to centralized databases containing sensitive information. These breaches can lead to significant financial loss, legal consequences, and damage to reputation.

Federated learning mitigates data breaches by keeping data decentralized on local devices, reducing centralized storage risks. Differential privacy further enhances security by adding noise to the data and model updates, ensuring that even if some information is accessed, it remains anonymized and less useful to attackers.

II. Membership Inference Attacks³: Membership inference attacks aim to determine whether a specific individual's data was included in the training dataset of a machine learning model. This can compromise user privacy and reveal sensitive information about the individual.

Differential privacy provides robust defense mechanisms against membership inference attacks by ensuring that the contribution of any single data point to the final model is obscured through the addition of noise.

III. Model Inversion Attacks⁴: Model inversion attacks allow adversaries to reconstruct or infer sensitive information about individuals from the outputs or parameters of a trained machine learning model. This can expose private details that were part of the training data.

Differential privacy mitigates model inversion attacks by introducing noise that makes it difficult for attackers to accurately infer individual data points from model outputs. Federated learning further protects against these attacks by decentralizing the training process and keeping the raw data on local devices, thus reducing the chances of sensitive information being reconstructed from model updates.

IV. Linkage Attacks⁵: Linkage attacks occur when attackers combine datasets from different sources to re-identify individuals or infer sensitive information. By correlating anonymized data with other available datasets, adversaries can reveal identities and other private details that were assumed to be protected.

Federated learning mitigates linkage attacks by keeping raw data decentralized, making it impossible for attackers to obtain data sets needed for such attacks. Differential privacy adds noise to the data and model updates, ensuring that the information about any single data point is obscured, making it difficult for attackers to accurately link records from different datasets and re-identify individuals.

³ Chen, J., Wang, W. H., & Shi, X. (2021). Differential Privacy Protection Against Membership Inference Attack on Machine Learning for Genomic Data. In *Pacific Symposium on Biocomputing* (Vol. 26, pp. 26–37). https://psb.stanford.edu/psb-online/proceedings/psb21/chen_j.pdf

⁴ Zhang, Q., Ma, J., Xiao, Y., Lou, J., & Xiong, L. (2020). *Broadening Differential Privacy for Deep Learning Against Model Inversion Attacks*. <https://doi.org/10.1109/bigdata50022.2020.9378274>

⁵ *Linkage attack - MOSTLY AI*. (n.d.). MOSTLY AI. <https://mostly.ai/synthetic-data-dictionary/linkage-attack>

2. Related works

- I. "Protecting Data from all Parties: Combining Homomorphic Encryption and Differential Privacy in Federated Learning"⁶: This work proposed a framework that combines homomorphic encryption and differential privacy to ensure training data privacy in federated learning, addressing threats from both the aggregating server and participating clients.
- II. "Deep Learning with Differential Privacy"⁷ by Abadi. : This paper introduced techniques for training deep neural networks with differential privacy, including gradient clipping and noise addition. It laid the foundation for applying differential privacy to federated learning.
- III. Federated Learning with Formal Differential Privacy"⁸ by Brendan McMahan : This paper from Google proposed practical algorithms for federated learning with differential privacy, including the Federated Averaging algorithm and techniques for user-level and record-level differential privacy.

4. Implementation

Implementation code for this project github.com/ronnie-1947/federated-learning

In this project, we are exploring the integration of differential privacy and federated learning to develop a framework for incorporating both techniques into any machine learning model. For testing, we use the MNIST dataset, a widely-used dataset consisting of 70,000 images of handwritten digits, which is ideal for training image classification models. Our approach includes preprocessing the dataset, partitioning it for federated learning scenarios, and implementing a neural network model to classify the digit images. By integrating differential privacy and homomorphic encryption, we aim to enhance data privacy and security while maintaining high model performance within a federated learning environment.

⁶ *Protecting Data from all Parties: Combining Homomorphic Encryption and Differential Privacy in Federated Learning*. (n.d.). ar5iv. <https://ar5iv.labs.arxiv.org/html/2205.04330>

⁷ Abadi, M., Chu, A., Goodfellow, I., McMahan, H. B., Mironov, I., Talwar, K., & Zhang, L. (2016). *Deep Learning with Differential Privacy*. <https://doi.org/10.1145/2976749.2978318>

⁸ *Federated Learning with Formal Differential Privacy Guarantees*. (n.d.). <https://research.google/blog/federated-learning-with-formal-differential-privacy-guarantees/>

Our framework works as a client-server model, where clients and a central server collaboratively train a machine learning model while keeping data decentralized. Each client, which represents a local device or data holder, trains a local model on its subset of the data and periodically sends only the model updates (such as gradients or weights) to the central server. The server aggregates these updates to create a global model, which is then sent back to the clients for further training.

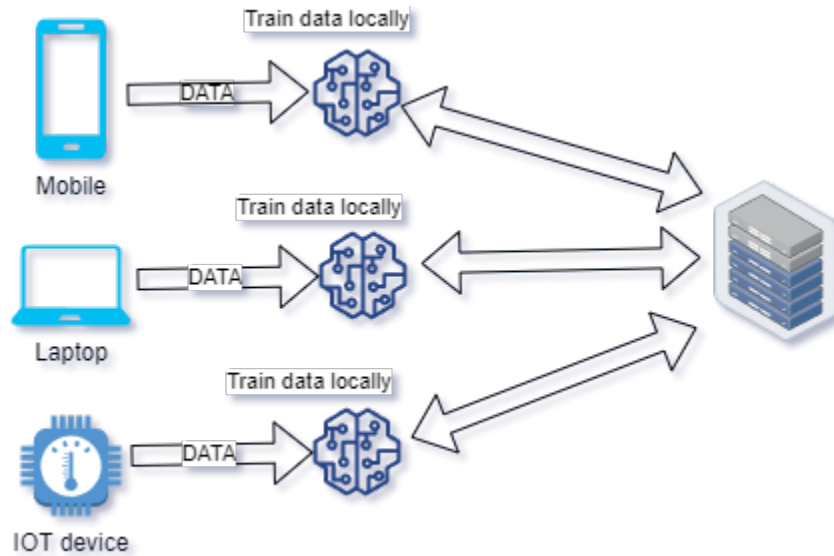


Fig 4.1 Federated learning client-server model

- I. **Data preparation:** In the data preparation step of our federated learning architecture, we start by separating the MNIST dataset into subsets, each representing a simulated client. This division enables us to effectively evaluate the federated learning process by simulating how data would be spread across several clients in a real-world setting. In reality, each client would already have its own training data, which would include multiple and perhaps heterogeneous datasets. By dividing the MNIST data into many client datasets, we can mimic the decentralized training environment and ensure that our framework appropriately represents the conditions under which federated learning functions.



Fig 4.2 Split dataset

For this purpose, we use a function called “prepare_dataset” from our library. This function takes any dataset and splits it into n parts for n clients in different ratios as needed (with equal parts as the default), effectively simulating a federated learning environment. Additionally, it utilizes PyTorch dataloaders to efficiently manage the data.

- II. **Initiate server:** Next step is to initiate the server. For this purpose, we use the `start_server` function from our federated library. This function accepts a server address (`host_addr`), the number of training rounds (`num_rounds`), and an optional strategy (`strategy`). If no strategy is provided, the function defaults to using the FedAvg (Federated Averaging) strategy. FedAvg⁹ is a commonly used algorithm in federated learning that combines model updates from multiple clients by averaging them. Within the function, if no custom strategy is specified, a weighted averaging function is defined to aggregate the accuracy metrics from each client based on the number of examples they used. This weighted average ensures that the global model's accuracy reflects the contributions of all clients proportionally. The function then configures the server with the specified number of rounds and chosen strategy, and starts the federated learning server at the given address using Flower (`fl`), a popular federated learning library.

⁹ McMahan, H. B., Moore, E., Ramage, D., Hampson, S., & Arcas, B. a. Y. (2016, February 17). Communication-Efficient Learning of Deep Networks from Decentralized Data. arXiv.org. <https://arxiv.org/abs/1602.05629>

```

from lib.federated import start_server

def main(cfg):
    start_server(host_addr='localhost:5050', num_rounds=2)

if __name__ == "__main__":
    main()

```

Fig 4.3 Initiate server

For reference, we used code from the Flower Framework repository:

<https://github.com/adap/flower/blob/main/examples/pytorch-from-centralized-to-federated/server.py>

- III. **Define model:** The next step is to define a machine learning model, which can be any model written in PyTorch. Our framework is designed to support any PyTorch-based model, ensuring flexibility and adaptability for various machine learning tasks. In the test code, we define a convolutional neural network (CNN) class `Net` for classifying the MNIST dataset. The model includes two convolutional layers, followed by max-pooling and three fully connected layers. The `train` function trains the model using the provided data loader, optimizer, and specified number of epochs. It leverages the cross-entropy loss function to compute the loss and performs backpropagation to update the model's parameters. The `test` function evaluates the model's performance by calculating the total loss and accuracy on the test dataset. During evaluation, the model is set to evaluation mode, and gradient calculations are disabled to save memory and computation.

For reference, we used code from the Flower Framework repository:

<https://github.com/adap/flower/blob/main/examples/flower-simulation-step-by-step-pytorch/Part-I/model.py>

Below is the code from our implementation, which defines a model in PyTorch and includes the train and test functions.

```

model.py > ...
1  import torch
2  import torch.nn as nn
3  import torch.nn.functional as F
4  from torch.utils.data import DataLoader
5
6  # Note the model and functions here defined do not have any FL-specific components.
7  class Net(nn.Module):
8      def __init__(self, num_classes: int) -> None:
9          super(Net, self).__init__()
10         self.conv1 = nn.Conv2d(1, 6, 5)
11         self.pool = nn.MaxPool2d(2, 2)
12         self.conv2 = nn.Conv2d(6, 16, 5)
13         self.fc1 = nn.Linear(16 * 4 * 4, 120)
14         self.fc2 = nn.Linear(120, 84)
15         self.fc3 = nn.Linear(84, num_classes)
16
17         def forward(self, x: torch.Tensor) -> torch.Tensor:
18             x = self.pool(F.relu(self.conv1(x)))
19             x = self.pool(F.relu(self.conv2(x)))
20             x = x.view(-1, 16 * 4 * 4)
21             x = F.relu(self.fc1(x))
22             x = F.relu(self.fc2(x))
23             x = self.fc3(x)
24             return x
25
26     def train(model: Net, trainloader:DataLoader, optimizer, epochs:int, device: str):
27         criterion = torch.nn.CrossEntropyLoss()
28         model.train()
29         model.to(device)
30         for _ in range(epochs):
31             for images, labels in trainloader:
32                 images, labels = images.to(device), labels.to(device)
33                 optimizer.zero_grad()
34                 loss = criterion(model(images), labels)
35                 loss.backward()
36                 optimizer.step()
37
38
39     def test(model:Net, testloader:DataLoader, device:str):
40         try:
41             criterion = torch.nn.CrossEntropyLoss()
42             correct, loss = 0, 0.0
43             model.eval()
44             model.to(device)
45             with torch.no_grad():
46                 for data in testloader:
47                     images, labels = data[0].to(device), data[1].to(device)
48                     outputs = model(images)
49                     loss += criterion(outputs, labels).item()
50                     _, predicted = torch.max(outputs.data, 1)
51                     correct += (predicted == labels).sum().item()
52             accuracy = correct / len(testloader.dataset)
53             return loss, accuracy
54         except Exception as e:
55             print('EXCEPTION ERROR',e)

```

Fig 4.4 Define ML model

- IV. Initiate client :** After model initialization, we are ready to prepare the client for federated learning. In our framework, we achieve this by utilizing the `fl_client` function, which integrates various components required for the client's participation in the federated learning process. The function takes several parameters, including the initialized model, the number of training epochs, the optimizer, the training and validation data loaders specific to the client, and the device configuration (e.g., CPU or GPU).

The `fl_client` function encapsulates these parameters into a client instance that is compatible with the Flower federated learning framework. It ensures that each client trains the model on its local data for the specified number of epochs, using the provided optimizer. The `train` and `test` functions are passed to handle the training and evaluation processes, respectively, ensuring that the client's model updates are correctly computed and assessed. This setup allows each client to independently train its local model and send updates to the central server, which aggregates these updates to improve the global model.

```
client_id = args.client_id

# Download Dataset
trainset, testset = get_mnist()

# Prepare dataset
trainloaders, valloaders, testloader = prepare_dataset(trainset=trainset, testset=testset, batch_size=32, num_partitions=3)

# Prepare federated client
model = Net(num_classes=10)
optimizer = torch.optim.SGD(model.parameters(), lr=0.1, momentum=0.9)

# Initiate client
client = fl_client(
    model=model,
    epochs=10,
    optimizer=optimizer,
    trainloader=trainloaders[int(client_id)],
    valloader=valloaders[int(client_id)],
    train=train,
    test=test,
    device=device).to_client()
```

Fig 4.5 Initiate client

For reference, we used code from the Flower Framework repository:

<https://github.com/adap/flower/tree/main/examples/pytorch-from-centralized-to-federated>

- V. Incorporate differential privacy:** The next stage in our framework is to create a privacy engine that incorporates differential privacy into the training process. We use the Opacus framework, a Facebook-developed library for training PyTorch models with differential privacy. Opacus assures that the model complies with privacy standards by adding noise to the gradients during training and clipping them to a specific norm, offering mathematical guarantees that individual data points cannot be easily retrieved from the trained model.

In the test code, we initialize a PrivacyEngine object from Opacus. This privacy engine is then used to modify the model, optimizer, and data loader to support differential privacy. The `make_private` method of PrivacyEngine is called with the model, optimizer, and data loader as arguments, along with two important hyperparameters: `noise_multiplier` and `max_grad_norm`. The `noise_multiplier` controls how much noise is introduced to the gradients to mask the effect of individual data points, while `max_grad_norm` specifies a threshold for gradient clipping, preventing any single gradient from having too much influence on the model update.

```
# Prepare dataset
trainloaders, valloaders, testloader = prepare_dataset(trainset=trainset , testset=testset, batch_size=32, num_partitions=3)

# Prepare federated client
model = Net(num_classes=10)
optimizer = torch.optim.SGD(model.parameters(), lr=0.1, momentum=0.9)

privacy_engine = PrivacyEngine()
model, optimizer, data_loader = privacy_engine.make_private(
    module=model,
    optimizer=optimizer,
    data_loader=trainloaders[int(client_id)],
    noise_multiplier=1.1,
    max_grad_norm=1.0,
)

# Initiate client
client = fl_client(
    model=model,
    epochs=10,
    optimizer=optimizer,
    trainloader=data_loader,
    valloader=valloaders[int(client_id)],
    train=train,
    test=test,
    device=device).to_client()
```

Fig 4.6 Incorporate differential privacy

Reference Code: <https://opacus.ai/>

- VI. Train model:** The last step is to start training. We use the `start_client` function from our library. This function connects the prepared client to the federated learning server and begins the learning process.

```
from lib.federated import fl_client, start_client

# Initiate client
client = fl_client(
    model=model,
    epochs=10,
    optimizer=optimizer,
    trainloader=data_loader,
    valloader=valloaders[int(client_id)],
    train=train,
    test=test,
    device=device).to_client()

# Connect to server and start learning
start_client(server_address='localhost:5050', client=client)
```

Fig 4.7 Model training

To automate the process, we wrote a shell script to train three clients.

```
$ run.sh
1  #!/bin/bash
2
3  echo "Starting server"
4
5  python server.py &
6  sleep 3
7
8  for i in $(seq 0 2); do
9      echo "Starting client $i"
10     python client.py --client-id $i &
11 done
12
13 # This will allow you to use CTRL+C to stop all background processes
14 trap "trap - SIGTERM && kill -- -$$" SIGINT SIGTERM
15 # Wait for all background processes to complete
16 wait
```

Fig 4.8 Shell script

5. Results

To test this federated learning setup, we trained a model across four rounds with a carefully selected configuration involving 10 clients. Each round required all 10 clients to be available and actively participate in both the fitting and evaluation processes. The model was trained with a batch size of 32, and the task involved classifying data into 10 output classes, representing the digits 0 through 9 in the MNIST dataset. The training setup included a learning rate of 0.1 and a momentum of 0.9, with each client running 100 local epochs per round. These parameters were chosen to ensure that the model effectively adapted to the data distributed across the clients.

```
! base.yaml
num_round: 4
num_clients: 10
min_available_clients: 10
batch_size: 32
num_classes: 10 # 10 output digits
num_clients_per_round_fit: 10
num_clients_per_round_eval: 10
config_fit:
  lr: 0.1
  momentum: 0.9
  local_epochs: 100
```

Fig 5.1 Federated learning configuration

Throughout the four training rounds, the model demonstrated a steady reduction in loss values, beginning at 43.68 in the first round and decreasing to 39.62 by the fourth round. This consistent decline in loss indicates that the model was successfully minimizing errors as the training progressed.

```

[2024-08-09 10:28:15,968][flwr][INFO] - History (loss, distributed):
INFO : round 1: 43.68376500606537
[2024-08-09 10:28:15,968][flwr][INFO] - round 1: 43.68376500606537
INFO : round 2: 43.48970377445221
[2024-08-09 10:28:15,969][flwr][INFO] - round 2: 43.48970377445221
INFO : round 3: 41.16119018793106
[2024-08-09 10:28:15,969][flwr][INFO] - round 3: 41.16119018793106
INFO : round 4: 39.62039366960526
[2024-08-09 10:28:15,969][flwr][INFO] - round 4: 39.62039366960526

```

Fig 5.2 Loss values

In terms of accuracy, the model showed gradual improvements over the rounds. Starting with a low accuracy of 4.77% in the first round, the accuracy improved to 8.32% in the second round, slightly increased to 8.7% in the third round, and reached 11.65% by the fourth round. While these accuracy gains are modest, they reflect the model's progressive enhancement in correctly predicting the output classes.

```

[flwr][INFO] - {'accuracy': [(1, 0.04766666666666667),
(2, 0.08316666666666667),
[flwr][INFO] - (2, 0.08316666666666667),
(3, 0.087),
[flwr][INFO] - (3, 0.087),
(4, 0.1165)]}
[flwr][INFO] - (4, 0.1165)]}

```

Fig 5.3 Model accuracy

Overall, with this training configuration, including client number local epochs, and learning parameters, showed ideal patterns for loss reduction and accuracy improvement. However, additional training or modifications to the model and training parameters are required to produce more performance gains.

6. Challenges

- I. **Hardware and GPU Limitations:** One of the significant challenges in this project was the limitation of available hardware and GPUs for model training. Federated learning requires substantial computational resources, especially when dealing with model training across multiple clients. Unfortunately, the available hardware was insufficient to meet these requirements efficiently.

- II. Lack of Proper Documentation:** Another major challenge we faced was the lack of comprehensive documentation for some of the packages used in the project, particularly PySyft. As a result, we had to experiment with multiple libraries and code repositories to complete the project. This significantly increased the development time, as we had to invest considerable effort in understanding and adapting various tools and resources to meet the project's requirements.

7. Conclusion

In conclusion, our project demonstrates the effective integration of differential privacy and federated learning to create a secure and privacy-preserving machine learning framework using the PyTorch library. By testing on the MNIST dataset, we were able to simulate real-world federated learning scenarios, showing how decentralized training and privacy-enhancing techniques can be implemented. Our framework includes essential steps such as data preparation, model definition, client-server setup, and the incorporation of differential privacy through the Opacus library.

The combination of federated learning and differential privacy addresses significant privacy concerns in machine learning, providing a scalable solution that maintains the confidentiality of individual datasets while leveraging the collective power of distributed data. By enabling model training across multiple clients, our framework paves the way for broader adoption of privacy-preserving machine learning techniques in various domains, such as healthcare, finance, and beyond. This project highlights the potential of advanced privacy technologies to enhance data security and trust in collaborative AI systems.